

Course “Object-Oriented Programming in C++”**Week №8*****Homework*****Exercise 1.**

Implement a stack in the form of singly-linked list. Implement the typical operations with a stack. If a stack is overflowed, it is necessary to increase its size.

Exercise 2.

Add to an existing class of singly-linked list: an operation cloning list (the function should return the address of the head of the cloned list), overloading the operator + (the operator has to return the address of the head of a new list containing elements of both lists, for which this operator was called), overloading the operator * (the operator should return the address of the head of a new list containing only common elements of both lists for which this operator was called).

Exercise 3.

Implement a class template "Queue" based on a doubly-linked list.

Exercise 4.

Add to an existing class of doubly-linked list: an operation cloning list (the function should return the address of the head of the cloned list), overloading the operator + (the operator has to return the address of the head of a new list containing

elements of both lists, for which this operator was called), overloading the operator $*$ (the operator should return the address of the head of a new list containing only common elements of both lists for which this operator was called).

Exercise 5.

Create a template class-container Array, which is an array that allows you to store objects of specified type. This class must be implemented using a doubly-linked list. This class must implement the following functions:

- a. GetSize — to get the size of the array (number of elements, for which memory is allocated)
- b. SetSize(int size, int grow = 1) — to set the size of the array (if the size parameter is greater than the previous size of the array, the additional memory block is allocated, if it's not greater, the "extra" items are lost, and the memory is released) ; the grow parameter defines for how many items you need to allocate memory if the number of items exceeds the current size of the array. For example, SetSize (5, 5); It means that when we add the 6 — element, an array size becomes equal to 10 , when we add the 11 — th an array size becomes equal to 15 and so on.
- c. GetUpperBound — get the last acceptable index in the array. For example, if the array size is 10, you add to it 4 elements, the function will return 3.
- d. IsEmpty — is an array empty?
- e. FreeExtra — to free any extra memory (above the last allowable index)
- f. RemoveAll — to delete everything
- g. GetAt — to get a specific element (by index)

- h. SetAt — to set a new value for a specific item (the index of the element must be less than the current size of the array)
- i. operator [] — to implement two previous functions
- j. Add — to add an element to the array (if it's necessary, the array is incremented by the value grow of SetSize function)
- k. Append — "Addition" of two arrays
- l. operator =
- m. GetData — get the address of an array with data
- n. InsertAt — insert the element (s) to a specified position
- o. RemoveAt — delete the item (s) from a specified position

Exercise 6.

Create a class template Matrix. It is necessary to implement the dynamic allocation of memory, to clean up, to fill the matrix from the keyboard, to fill with random values, to display the matrix, to perform arithmetic operations using the overloading operators (+ , - , * , /) , to search for the maximum and minimum element.